

2.1 Ingeniería de Requisitos

Definición de Ingeniería de Requisitos

“La ingeniería de requisitos comprende los procesos relacionados con la identificación, documentación, análisis, validación y gestión de requisitos de sistemas y software.” [1]

La ingeniería de requisitos ayuda a identificar lo que realmente necesitan los usuarios antes de comenzar a desarrollar el software. Gracias a este proceso, es posible definir de mejor manera las funciones y características que tendrá el sistema. También permite que exista una mejor comunicación entre los clientes y los desarrolladores, evitando confusiones o errores durante el proyecto.

Ejemplo: En la red social para mascotas, la ingeniería de requisitos permitirá identificar funciones como el registro de mascotas, adopciones y servicios veterinarios.

Objetivos de la Ingeniería de Requisitos

“Los procesos de requisitos buscan asegurar que las necesidades y expectativas de los stakeholders sean correctamente definidas y gestionadas.” [2]

Los objetivos de la ingeniería de requisitos son entender lo que necesitan los usuarios y convertir esas necesidades en requisitos claros para el desarrollo del sistema. Esto ayuda a evitar errores o cambios innecesarios que pueden generar más tiempo y gastos durante el proyecto. Además, permite que el software final cumpla de mejor manera con las expectativas de las personas que van a utilizarlo.

Ejemplo: La plataforma deberá considerar las necesidades de dueños de mascotas, veterinarios y refugios de animales.

Importancia de la Ingeniería de Requisitos en proyectos software

“Los requisitos bien definidos son fundamentales para el éxito de los proyectos de software.” [6]

La ingeniería de requisitos es importante porque permite tener una idea clara de lo que se necesita desarrollar en el software desde el inicio del proyecto. Cuando los requisitos no están bien definidos, pueden aparecer problemas como retrasos, gastos adicionales o errores en el sistema. Por esta razón, este proceso ayuda a mejorar la organización y la calidad del proyecto durante su desarrollo.

Ejemplo: Definir correctamente las funcionalidades de adopción y búsqueda de mascotas evitará errores durante el desarrollo de la plataforma.

2.2 Requisito de Software

Definición de requisito de software

“Un requisito es una condición o capacidad necesaria para satisfacer una necesidad de un usuario o contrato.” [1]

Los requisitos de software permiten definir las funciones y condiciones que debe cumplir un sistema para satisfacer las necesidades de los usuarios. Estos ayudan a orientar el diseño, desarrollo y evaluación del software durante todo el proyecto. Además, facilitan que tanto los clientes como los desarrolladores tengan claro lo que se espera del sistema.

Ejemplo: El sistema deberá permitir el registro de usuarios y mascotas dentro de la plataforma.

Cadena necesidad → meta → requisito

“Las necesidades de los stakeholders se transforman en metas y posteriormente en requisitos del sistema.” [5]

La cadena necesidad-meta-requisito ayuda a transformar las necesidades de los usuarios en funciones específicas dentro del sistema. Primero se identifica el problema o necesidad que tiene el usuario, después se define una meta para resolverlo y finalmente se establece un requisito claro para el software. Esto permite que el sistema desarrollado tenga relación con lo que realmente esperan y necesitan los usuarios.

Ejemplo: Necesidad: encontrar mascotas para adopción. Meta: facilitar adopciones responsables. Requisito: el sistema permitirá publicar mascotas disponibles para adopción.

Características de un buen requisito

“Un buen requisito debe ser claro, consistente, verificable y sin ambigüedades.” [1]

Las características de un buen requisito permiten que todas las personas involucradas en el proyecto puedan entenderlo de forma clara. Cuando un requisito está bien redactado, se reducen las confusiones y resulta más fácil desarrollarlo dentro del sistema. Además, esto ayuda a comprobar después si el software cumple correctamente con lo que fue solicitado.

Ejemplo: “El usuario podrá registrar hasta cinco mascotas por cuenta” es un requisito claro y específico.

Requisitos verificables y cuantificables

“Los requisitos deben formularse de manera que puedan ser verificados y medidos.” [8]

Los requisitos verificables ayudan a comprobar de manera clara si el sistema cumple con lo que fue solicitado. Por otro lado, los requisitos cuantificables incluyen datos medibles, como tiempo de respuesta, rendimiento o cantidad de usuarios soportados. Esto facilita realizar pruebas y validar que el software funcione correctamente durante su desarrollo.

Ejemplo: La plataforma deberá cargar las publicaciones en menos de tres segundos.

2.3 Tipos de Requisitos

Requisitos funcionales

“Los requisitos funcionales describen funciones o servicios que el sistema debe proporcionar.” [8]

Los requisitos funcionales describen las acciones y funciones que debe realizar el software dentro del sistema. Estos ayudan a definir las principales tareas y servicios que estarán disponibles para los usuarios. Además, sirven como una guía importante para el diseño y desarrollo del proyecto.

Ejemplo: El sistema permitirá buscar mascotas según raza, tamaño y ubicación.

Requisitos no funcionales

“Los requisitos no funcionales describen atributos de calidad y restricciones del sistema.” [4]

Los requisitos no funcionales definen aspectos relacionados con la calidad del software, como la seguridad, el rendimiento y la facilidad de uso. Estos requisitos ayudan a que el sistema funcione de manera más eficiente y brinde una mejor experiencia a los usuarios. Además, permiten mantener un funcionamiento estable y confiable durante el uso de la plataforma.

Ejemplo: La plataforma deberá estar disponible el 99 % del tiempo.

Requisitos de producto

“Los requisitos de producto describen propiedades y características del sistema desarrollado.” [5]

Los requisitos de producto describen las características técnicas y funcionales que debe tener el software. Estos ayudan a definir cómo funcionará el sistema y qué características ofrecerá a los usuarios una vez que esté implementado. Además, permiten establecer aspectos relacionados con la calidad y el correcto funcionamiento de la plataforma.

Ejemplo: La aplicación deberá ser compatible con dispositivos Android e iOS.

Requisitos de proceso

“Los requisitos de proceso definen condiciones relacionadas con el desarrollo y mantenimiento del sistema.” [5]

Los requisitos de proceso definen las normas, métodos y actividades que se deben seguir durante el desarrollo del software. Estos ayudan a organizar mejor el trabajo del equipo y a mantener buenas prácticas durante el proyecto. Además, permiten tener un mayor control y asegurar la calidad en cada etapa del desarrollo.

Ejemplo: El desarrollo del sistema utilizará metodología ágil Scrum.

Propiedades emergentes

“Las propiedades emergentes surgen de la interacción entre los componentes del sistema.” [5]

Las propiedades emergentes son características o comportamientos que aparecen cuando todas las partes del sistema funcionan en conjunto. Estas no dependen solamente de un componente específico, sino de la interacción entre todos los elementos del sistema. Además, permiten evaluar aspectos importantes como el rendimiento y la confiabilidad general del software.

Ejemplo: La estabilidad de la plataforma dependerá de la interacción entre servidores, base de datos y usuarios conectados.

Requisitos cuantificables

“Los requisitos cuantificables incluyen métricas y valores medibles.” [1]

Los requisitos cuantificables permiten definir condiciones específicas mediante valores numéricos o métricas medibles. Esto ayuda a comprobar de forma más sencilla si el sistema cumple correctamente con lo solicitado. Además, permiten establecer límites claros sobre el funcionamiento y rendimiento del software.

Ejemplo: El sistema soportará hasta 500 usuarios conectados simultáneamente.

Restricciones

“Las restricciones limitan las opciones de diseño o implementación del sistema.” [8]

Las restricciones son condiciones o límites que deben tomarse en cuenta durante el desarrollo del software. Estas pueden estar relacionadas con el tiempo disponible, el presupuesto, las tecnologías utilizadas o ciertas normas que deben cumplirse. Además, ayudan a definir hasta dónde puede llegar el proyecto y qué aspectos deben respetarse durante su desarrollo.

Ejemplo: Solo usuarios registrados podrán publicar información sobre mascotas.

Referencias

[1] IEEE, ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering, 2nd ed., 2018.

[2] IEEE, ISO/IEC/IEEE 15288:2023 Systems and software engineering — System life cycle processes, 2023.

[3] IEEE, ISO/IEC/IEEE 12207:2017 Systems and software engineering — Software life cycle processes, 2017.

[4] ISO/IEC 25010:2023 Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model, 2023.

- [5] K. Pohl, Requirements Engineering: Fundamentals, Principles and Techniques, 2025.
- [6] SWEBOK Guide V4.0, Guide to the Software Engineering Body of Knowledge, IEEE Computer Society, 2024.
- [7] Project Management Institute, A Guide to the Project Management Body of Knowledge (PMBOK Guide), 7th ed., 2021.
- [8] IREB, CPRE Foundation Level Syllabus v3.1, International Requirements Engineering Board, 2023.